

One Token Per Frame: Reconsidering Visual Bandwidth in World Models for VLA Policy

Zuojin Tang¹ Shengchao Yuan² Xiaoxin Bai³ Zhiyuan Jing⁵
De Ma¹ Gang Pan^{1*} Bin Liu^{4*}

¹Zhejiang University ²Central South University ³Harbin Institute of Technology

⁴Embodied Intelligence General Platform Laboratory, Chery Auto

⁵E-surfing Digital Life Technology Co., Ltd., China Telecom

*Corresponding Author

Abstract

Vision-language-action (VLA) models increasingly rely on auxiliary world modules to plan over long horizons, yet how such modules should be parameterized on top of a pretrained VLA remains an open design question. Existing world-model-augmented VLAs typically pass the per-frame visual stream into the world module at high visual bandwidth and treat its rollout as a side product of action prediction; under a constrained adaptation budget on a frozen backbone, this leaves both the per-frame representation and the latent action coupling under-examined. We introduce **OneWM-VLA**, which compresses each view into a single semantic token per frame through an Adaptive Attention Pooling, and produces the resulting latent stream and the action trajectory under a single flow-matching objective rather than connecting them through a separate decoder. Empirically, we find that per-frame visual bandwidth can be reduced to a single token without compromising long-horizon performance under our setup. Trained with 14.71M LoRA parameters on a π_0 (2B) backbone, OneWM-VLA improves the average success rate from 47.9% to 61.3% on MetaWorld MT50, reaches 95.6% on LIBERO-Long (vs. 85.2% for π_0), and reaches 60.0% on the long-horizon deformable task *Fold Cloth* on a real Piper arm (vs. 20.0% for π_0).

1 Introduction

Vision-language-action (VLA) models [7, 52, 27, 26, 14, 6, 23] have become a dominant paradigm for robot learning, mapping language instructions and visual observations directly to actions. While such policies excel at short-horizon imitation, they fall short on tasks that require anticipating how the scene will change under future actions, leading to error accumulation along the planning horizon. A natural remedy is to attach a forward model of environment dynamics to the policy [10, 47, 21], predicting future video frames as auxiliary supervision or as goals for action generation. While future-frame prediction is an expressive grounding signal, the per-step compute of pixel-level rollouts grows quickly with the horizon, and the prediction target is largely dominated by photometric detail rather than control-relevant dynamics, making it costly to scale when only a small adaptation budget is available on top of a frozen backbone.

Latent-space world models offer a complementary route. Recurrent state-space models [18, 19] and joint-embedding predictors [28, 3] show that compact learned representations support long-horizon reasoning at much lower per-step cost than pixel-level rollouts, by predicting in a learned representation rather than in pixel space. These methods have, however, largely been developed within their own reinforcement learning or self-supervised pipelines, and have not been carried over to the adaptation regime of a pretrained generative VLA, where the backbone is largely frozen

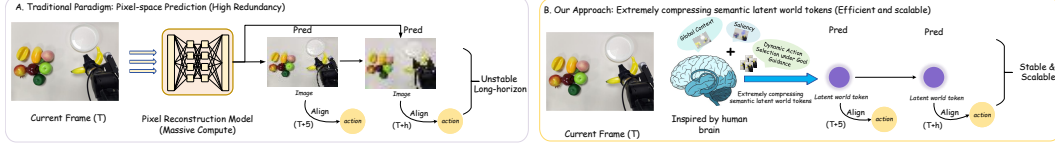


Figure 1: Motivation. OneWM-VLA represents each frame by a single semantic latent, keeping the per-step world-module budget horizon-invariant.

and only a small set of additional parameters is trained. This raises a basic design question for world-module-augmented VLAs in this regime: how much per-frame visual bandwidth a world module actually needs while still supporting long-horizon control, and how the resulting latent stream should be coupled to action generation.

In this paper, we propose OneWM-VLA (Fig. 1), a world-module-augmented VLA that instantiates the two design choices above as a bottleneck–rollout coupling. A lightweight Adaptive Attention Pooling distills each view into a single semantic latent token per frame, so that per-frame visual bandwidth is held at one token and the per-step world-module budget remains horizon-invariant. A joint flow-matching objective then generates the future latent stream and the future action trajectory under one model, so the predicted latent stream serves as a structural prior on the action trajectory rather than a side channel produced by a separate decoder. The bottleneck makes the joint objective tractable across the horizon, and the joint objective gives the bottleneck a control-relevant supervisory signal. We instantiate OneWM-VLA on top of the pretrained π_0 model [6] and train it with LoRA [20]; at inference, only the action branch is executed on the robot, while the latent tokens provide an internal auxiliary trajectory that helps structure action generation over the horizon.

We evaluate OneWM-VLA on simulated benchmarks and a real Piper arm. On MetaWorld MT50, OneWM-VLA improves the average success rate from 47.9% to 61.3%; on LIBERO-Long, it reaches 95.6%, above the 85.2% of π_0 . On the real Piper arm, it reaches 60.0% on the long-horizon deformable task *Fold Cloth*, compared with 20.0% for π_0 . To probe the design choice itself, we sweep the per-frame visual bandwidth from 1 to 12 tokens under a matched training budget; counter to the intuition that more visual tokens should help, success rate decreases monotonically as the bandwidth grows. This is consistent with the view that, in this adaptation regime, per-frame visual bandwidth is not where the difficulty primarily lies, and that a single semantic token per frame is a reasonable operating point rather than a degenerate one. Our contributions are as follows:

- We identify that pixel-level world modules scale poorly with the planning horizon when added on top of a pretrained VLA, and find that under our setup the per-frame visual bandwidth can be reduced to a single semantic token without compromising long-horizon control.
- We introduce OneWM-VLA, which realizes the *bottleneck–rollout coupling* through a single-token-per-frame Adaptive Attention Pooling and a joint flow-matching objective that generates the future latent stream and the action trajectory under one model.
- We report consistent gains over the π_0 backbone on LIBERO, MetaWorld MT50, and a real Piper arm, supported by a per-frame bandwidth sweep and a latent-supervision ablation in the adaptation regime studied here.

2 Related Work

2.1 Vision-Language-Action Models

Driven by the rapid progress of Multimodal Large Language Models [4, 30, 42, 2, 1] and the emergence of large-scale robot datasets, Vision-Language-Action (VLA) models [7, 52, 27, 23, 49, 12, 6, 39, 35, 8, 51, 26, 16, 41, 37] have become a dominant paradigm in robot learning, fine-tuning multimodal LLMs to map language instructions and visual observations directly to actions. To better capture the multi-modal nature of robot actions, a line of work [14, 6, 23, 33] replaces deterministic decoding with diffusion or flow matching. A complementary line of work extends VLAs with chain-of-thought reasoning, auxiliary heads, or multimodal traces [48, 44, 45, 22, 40, 29], which strengthens short-horizon planning. However, such policies remain largely reactive: they map current observations to actions without modeling how the scene will change under those actions, leaving

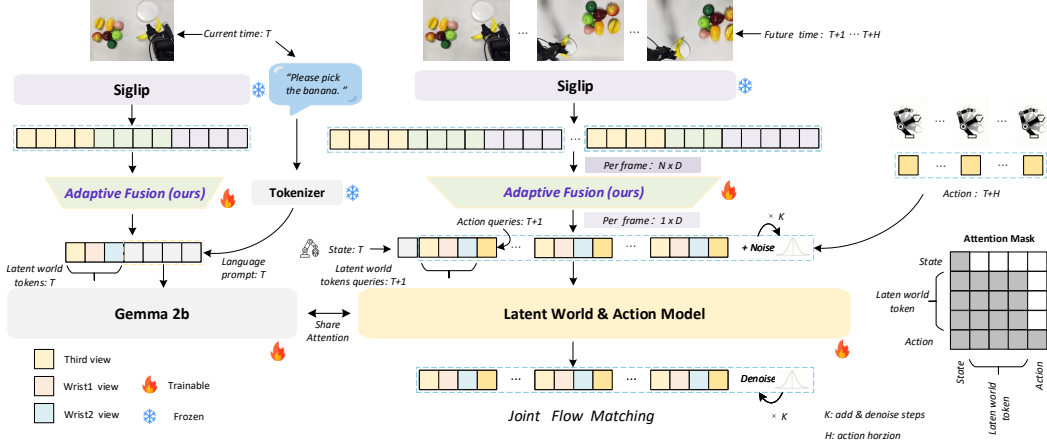


Figure 2: The OneWM-VLA Framework. Through Adaptive Attention Pooling (Adaptive Fusion), we distill multi-view visual features into a dense latent world token for each perspective. These tokens are aligned with action trajectories via Joint Flow Matching to ensure alignment between environment dynamics transitions and robot control.

long-horizon error accumulation unaddressed. In contrast, our work attaches an explicit forward model to the VLA, so the policy can anticipate future scene evolution while generating actions.

2.2 World Models for Policy Learning

Equipping policies with a forward model of environment dynamics has been a long-standing direction in model-based control. One line of work predicts in pixel space: autoregressive transformers or diffusion models synthesize future frames as goals or auxiliary supervision [10, 5, 11, 50, 43, 15, 24], and recent methods jointly generate future video and actions within a single framework [17, 46, 21, 9, 25, 47]. A complementary line of work predicts in a learned latent space: recurrent state-space models such as Dreamer [18, 19] roll out imagined trajectories in a low-dimensional posterior, JEPA [28, 3] predicts in a learned representation rather than in pixels, and sequence-modeling formulations [13, 38] show that compact latent representations can support long-horizon reasoning at much lower per-step cost. However, pixel-level approaches inherit the burden of dense reconstruction and per-step rollout cost grows quickly with the planning horizon, while latent-space approaches have largely been developed within their own RL or self-supervised pipelines and have not been carried over to a pretrained generative VLA. In contrast, our work attaches a compact latent world module on top of a pretrained VLA, and generates the latent rollout and the action trajectory under a single flow-matching objective rather than as two separate stages.

3 Preliminaries

3.1 Flow Matching

Flow matching [31] trains a continuous normalizing flow by regressing a vector field v_θ rather than simulating the underlying ODE. Given a probability path $p_t(x)$ that connects the data distribution p_0 and a prior p_1 , the objective is

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, x \sim p_t} [\|v_\theta(t, x) - u_t(x)\|^2], \quad (1)$$

where $u_t(x)$ is the target velocity field. Because $u_t(x)$ is in general intractable, we use the conditional flow matching (CFM) objective: with Gaussian conditional paths $p_t(x | x_1)$ and the closed-form conditional fields $u_t(x | x_1)$,

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, q(x_1), p_t(x|x_1)} [\|v_\theta(t, x) - u_t(x | x_1)\|^2]. \quad (2)$$

This objective is simulation-free at training time, and is the policy-side training objective inherited from the π_0 backbone.

4 Methodology

We instantiate the bottleneck–rollout coupling introduced in Sec. 1 as two interdependent components (Fig. 2). The bottleneck (Sec. 4.1) reduces each view to a single semantic latent token per frame through an Adaptive Attention Pooling, so that the per-step world-module token budget remains horizon-invariant. The rollout (Sec. 4.2) generates the future latent stream and the future action trajectory under a single flow-matching objective, so that the predicted latent stream serves as a structural prior on the action sequence rather than a side channel produced by a separate decoder. The bottleneck makes the joint objective tractable across the horizon, and the joint objective gives the bottleneck a control-relevant supervisory signal; we return to their empirical attribution in Sec. 5.

4.1 Per-Frame Compression via Adaptive Attention Pooling

Adaptive Attention Pooling reduces each view to a single token per frame in two stages: a token-level multi-strategy pooling and a view-level adaptive fusion. We process each camera independently and write $i \in \{r, w_1, w_2\}$ for the third-person view and the two wrist views.

Visual encoding. For each view i , we extract token features with the pretrained PaliGemma [4] encoder $\mathcal{E}_\phi$ from the input frames $\mathbf{I}_i \in \mathbb{R}^{B \times T \times H \times W \times C}$,

$$\mathbf{X}_i = \mathcal{E}_\phi(\mathbf{I}_i) \in \mathbb{R}^{B \times T \times N \times D}, \quad (3)$$

where $N=256$ is the number of visual tokens per frame and D is the hidden dimension. We write the n -th token of view i as $\mathbf{x}_i^{(n)} \in \mathbb{R}^D$.

Multi-strategy token pooling. We use three complementary scoring functions $\phi_m : \mathbb{R}^D \rightarrow \mathbb{R}$, $m \in \mathcal{M} = \{\text{MAX}, \text{SUM}, \text{LEARN}\}$, that score each token along different notions of saliency,

$$\phi_{\text{MAX}}(\mathbf{x}) = \max_d x^{(d)}, \quad \phi_{\text{SUM}}(\mathbf{x}) = \sum_{d=1}^D x^{(d)}, \quad \phi_{\text{LEARN}}(\mathbf{x}) = Q_\theta(\mathbf{x}), \quad (4)$$

corresponding to the peak channel response, the total channel response, and a learned task-aware response from a small view-specific MLP Q_θ . Each scoring function induces a softmax distribution over the N tokens at temperature τ ,

$$w_m^{(n)} = \frac{\exp(\phi_m(\mathbf{x}_i^{(n)})/\tau)}{\sum_{j=1}^N \exp(\phi_m(\mathbf{x}_i^{(j)})/\tau)}, \quad m \in \mathcal{M}, \quad (5)$$

which is then used to aggregate the N tokens along the token axis while keeping the feature dimension D intact,

$$\mathbf{p}_{\text{MAX}} = \max_n (w_{\text{MAX}}^{(n)} \mathbf{x}_i^{(n)}), \quad \mathbf{p}_m = \sum_{n=1}^N w_m^{(n)} \mathbf{x}_i^{(n)}, \quad m \in \{\text{SUM}, \text{LEARN}\}. \quad (6)$$

This produces three pooled tokens $\{\mathbf{p}_m\}_{m \in \mathcal{M}} \subset \mathbb{R}^D$ per frame, one per scoring strategy.

Adaptive view fusion. The three pooled tokens are combined into a single per-frame world token via a learnable convex combination. With trainable scalars $\alpha \in \mathbb{R}^{|\mathcal{M}|}$ and softmax-normalized weights β , the per-view world token $\mathbf{Z}_i \in \mathbb{R}^{B \times T \times 1 \times D}$ is

$$\mathbf{Z}_i = \sum_{m \in \mathcal{M}} \beta_m \mathbf{p}_m, \quad \beta_m = \frac{\exp(\alpha_m/\tau)}{\sum_{m' \in \mathcal{M}} \exp(\alpha_{m'}/\tau)}. \quad (7)$$

Because each view contributes one token per frame, the world-module token budget per step is independent of the visual resolution N , and the total token count fed into the joint generator scales as the planning horizon H multiplied by the number of views, rather than as $N \cdot H$. Two sources of adaptation enter the pooling. The token-level weights $w_m^{(n)}$ depend on the input through ϕ_m and vary from frame to frame, providing instance-specific feature selection. The view-level fusion weights β_m are learned scalars that are fixed at inference; we report their learned values in Sec. 5.3.

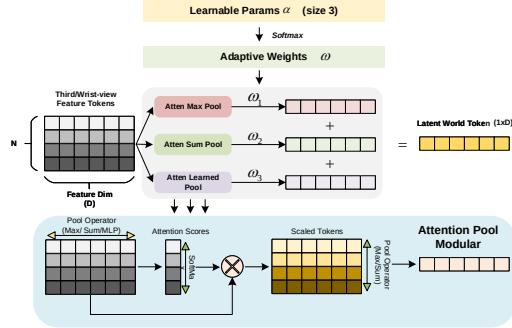


Figure 3: Adaptive attention pooling.

4.2 Joint Flow Matching over Latents and Actions

The bottleneck above turns each view into a per-frame latent token $\mathbf{Z}_i$. We now describe how the rollout couples the future latent stream with the action trajectory under a single flow-matching objective, so that the latent stream serves as a structural prior on the action sequence rather than a side channel.

Joint probability path. Let $a \in \mathbb{R}^{H \times D_a}$ denote the future action sequence and $z \in \mathbb{R}^{H \times D_z}$ the future latent world tokens. Following the optimal-transport (OT) displacement parameterization adopted by π_0 [6], we extend the same straight-line interpolation to the latent branch, so that both branches share an identical flow-time schedule. Given data samples (a, z) and noise $(\epsilon_a, \epsilon_z) \sim \mathcal{N}(0, I)$, the interpolated states at flow time t are

$$x_t^a = t\epsilon_a + (1-t)a, \quad x_t^z = t\epsilon_z + (1-t)z, \quad t \sim \text{Beta}(1.5, 1), \quad (8)$$

where the $\text{Beta}(1.5, 1)$ schedule places more probability mass near $t=0$, biasing training toward the data end of the path. The corresponding target velocity fields, which are constant along each interpolation, are

$$u_t^a = \epsilon_a - a, \quad u_t^z = \epsilon_z - z. \quad (9)$$

Multi-objective flow-matching loss. The transformer input is organized as a fused sequence interleaving current tokens $\{x_t^{z_r}, x_t^{z_{w_1}}, x_t^{z_{w_2}}, l_t, s_t\}$ (third-person view r , wrist views w_1, w_2 , language l , robot state s) with future noisy queries $\{x_{t+k}^{z_r}, x_{t+k}^{z_{w_1}}, x_{t+k}^{z_{w_2}}, x_{t+k}^a\}_{k=1}^h$. The model v_θ predicts a velocity field for both branches, trained with

$$\mathcal{L} = \lambda_a \mathbb{E}[\|v_\theta^a - u_t^a\|_1] + \sum_{i \in \{r, w_1, w_2\}} \lambda_i \mathbb{E}[\|v_\theta^z - u_t^z\|_1]. \quad (10)$$

Because the latent and action branches share the same generator and the same flow time, their predicted velocities v_θ^z and v_θ^a co-evolve through self-attention during both training and ODE-based sampling, rather than being connected by a post-hoc auxiliary loss. We examine the role of this coupling through a controlled ablation in Sec. 5.5. At inference, all modalities are initialized from Gaussian noise and jointly denoised, with only the action stream executed on the robot.

5 Experiments

We evaluate OneWM-VLA on simulated benchmarks and a real Piper arm, organized around two questions that follow from the design described in Sec. 4: whether the per-frame visual bandwidth can in fact be reduced to a single semantic token under our setup, and how each ingredient of the bottleneck–rollout coupling contributes to the resulting performance. We address these questions through five experimental probes:

- **Q1** (Sec. 5.2): Is a single per-frame token enough for long-horizon control?
- **Q2** (Sec. 5.3): Does the adaptive design of the pooling matter?
- **Q3** (Sec. 5.4): Should compression happen in semantic or in pixel space?
- **Q4** (Sec. 5.5): How much of the gain can be attributed to the joint flow-matching objective rather than to the added latent capacity alone?
- **Q5** (Sec. 5.6): How does OneWM-VLA scale with the planning horizon, and where does its advantage become most pronounced?

We treat all experimental conclusions as scoped to the training setup and manipulation suites considered, and refrain from extrapolating beyond them. Tab. 1, Tab. 2, and Tab. 3 report the main numbers; Q1–Q5 then dissect where these numbers come from.

Benchmarks. **MetaWorld MT50** [34] provides 50 manipulation tasks grouped into four difficulty tiers (Easy, Medium, Hard, Very Hard; Appendix A); we use it as the primary long-horizon testbed, with planning horizons up to $H=30$. **LIBERO** [32] provides four task suites (Spatial, Object, Goal, Long), with the Long suite targeting extended-horizon manipulation. **Real-world Piper.** A 6-DoF arm with a parallel gripper performs three tasks: *Pick Banana* (rigid grasping), *Fold Cloth* (long-horizon deformable manipulation), and *Pull Drawer* (articulated object). Each task is evaluated with 20 trials under clean conditions, with an additional perceptual-robustness check under lighting shifts, positional perturbations, and 4–6 unseen distractors.

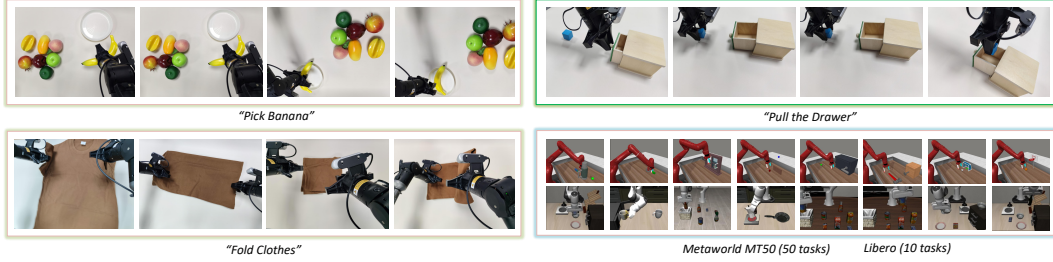


Figure 4: Evaluation suites used in this work: the LIBERO and MetaWorld MT50 simulation benchmarks, and a real Piper robot arm.

Table 1: Main results on MetaWorld MT50. Success rates (%) across difficulty tiers and planning horizons H . Δ_{π_0} and $\Delta_{\pi_{0.5}}$ denote the absolute gain over π_0 and $\pi_{0.5}$.

Method	Horizon	Easy	Medium	Hard	Very Hard	Avg %	Δ_{π_0}	$\Delta_{\pi_{0.5}}$
Diff. Policy	5	8.21	0.00	0.00	0.00	2.05	–	–
ACT	5	31.43	11.82	0.00	4.20	10.81	–	–
π_0	5	77.50	36.36	24.00	38.00	43.97	–	–
$\pi_{0.5}$	5	66.07	30.91	22.00	34.00	38.25	–	–
OneWM-VLA	5	79.29	41.82	64.00	60.00	61.28	+17.31	+23.03
Diff. Policy	10	9.64	0.00	0.00	0.00	3.45	–	–
ACT	10	47.50	7.37	0.00	4.00	14.72	–	–
π_0	10	77.50	30.91	30.00	48.00	46.60	–	–
$\pi_{0.5}$	10	73.21	28.18	20.00	16.00	38.84	–	–
OneWM-VLA	10	78.57	31.82	58.00	40.00	52.10	+5.50	+13.26
Diff. Policy	25	8.93	0.00	0.00	0.00	1.23	–	–
ACT	25	31.43	0.00	0.00	0.00	16.53	–	–
π_0	25	73.93	27.27	36.00	22.00	39.80	–	–
$\pi_{0.5}$	25	70.71	26.36	16.00	28.00	35.26	–	–
OneWM-VLA	25	74.64	30.00	26.00	54.00	46.16	+6.36	+10.90
Diff. Policy	30	6.43	0.00	0.00	0.00	1.60	–	–
ACT	30	31.43	0.00	0.00	0.00	7.86	–	–
π_0	30	80.00	35.45	32.00	4.00	37.98	–	–
$\pi_{0.5}$	30	63.57	31.82	8.00	4.00	26.83	–	–
OneWM-VLA	30	72.86	33.64	46.00	60.00	53.13	+15.15	+26.30

Implementation. We instantiate OneWM-VLA on top of π_0 [6], with PaliGemma-2B as the vision-language backbone and Gemma-300M as the joint latent-and-action expert. Both are trained with LoRA [20]. Flow matching uses 10 ODE steps at inference, with loss weights $\lambda_a=1.0$ and $\lambda_r=\lambda_{w_1}=\lambda_{w_2}=0.1$. We train for 30K steps on 8 NVIDIA A800 GPUs. On MetaWorld we evaluate $H \in \{5, 10, 25, 30\}$. On LIBERO we use a single training checkpoint (training action horizon 20) for all four suites and tune only the inference action horizon and the replan step per suite on a small validation grid (Appendix D). **Baselines.** We compare against π_0 (2B) and the larger $\pi_{0.5}$ (3B) as the primary VLA references, and additionally report standard reference points (Diffusion Policy, ACT, Octo, OpenVLA, SpatialVLA, CoT-VLA, WorldVLA, π_0 -FAST, SmolVLA, GR00T-N1, DreamVLA) where available.

5.1 Main Results

MetaWorld MT50. Tab. 1 reports four planning horizons. OneWM-VLA improves over π_0 and $\pi_{0.5}$ at every horizon, and the gap widens with H . At $H=30$, OneWM-VLA reaches 53.13% on average, π_0 drops to 37.98%, and $\pi_{0.5}$ to 26.83%. The improvement is concentrated on the Hard and Very Hard tiers.

LIBERO. Tab. 2 reports the four suites. OneWM-VLA reaches 98.1% on average, above $\pi_{0.5}$ (96.85%). On the Long suite, OneWM-VLA reaches 95.6%, +10.4 over π_0 and +3.2 over $\pi_{0.5}$. All

Table 2: Main results on LIBERO. Success rates (%) on the four suites. Inference-time configuration is detailed in Appendix D.

Method	Spatial	Object	Goal	Long	Avg%
Diffusion Policy [14]	78.3	85.7	88.4	68.3	72.4
Octo [41]	78.9	84.6	89.9	79.2	76.5
OpenVLA [27]	84.7	88.2	92.5	78.6	78.1
SpatialVLA [35]	88.2	89.9	92.5	79.2	78.1
CoT-VLA [48]	81.1	87.5	91.6	87.6	79.5
WorldVLA [10]	87.6	96.2	83.4	60.0	81.8
SmolVLA [37]	93.0	94.0	91.0	77.0	88.8
GR00T-N1 [5]	94.4	97.6	93.0	90.6	93.9
DreamVLA [47]	97.5	94.0	89.5	89.5	92.6
π_0 [6]	96.8	98.8	95.8	85.2	94.1
$\pi_{0.5}$ [23]	98.8	98.2	98.0	92.4	96.8
OneWM-VLA	98.2	99.6	99.0	95.6	98.1

Table 3: Real-world Piper manipulation results. 20 trials per task, under clean conditions and observation noise (lighting shifts, positional perturbations, unseen distractors). Both training and inference use action chunk size 50.

Method	P.Banana	F.Cloth	P.Drawer	Avg%
<i>Clean conditions</i>				
π_0 [6]	100.0	20.0	30.0	50.0
$\pi_{0.5}$ [23]	100.0	25.0	50.0	58.3
OneWM-VLA	100.0	60.0	55.0	71.7
<i>With observation noise</i>				
π_0 [6]	65.0	0.0	10.0	25.0
$\pi_{0.5}$ [23]	60.0	10.0	25.0	31.7
OneWM-VLA	75.0	40.0	35.0	50.0

four suites use the same training checkpoint; only the inference action horizon and the replan step are tuned per suite (Appendix D).

Real-world Piper. Tab. 3 reports results on the physical robot. Among the three tasks, *Fold Cloth* is the most demanding: the cloth state evolves continuously under contact, and the policy must remain stable across a long sequence of fine-grained interactions. Under clean conditions, OneWM-VLA reaches 71.7% on average, +21.7 over π_0 and +13.4 over $\pi_{0.5}$, with the largest absolute gain on *Fold Cloth* (60.0% vs. π_0 at 20.0% and $\pi_{0.5}$ at 25.0%). Under perceptual disturbances (lighting shifts, positional perturbations, and unseen distractors), OneWM-VLA reaches 50.0% on average, with a notably wider gap to π_0 on *Fold Cloth* (40% vs. 0%); we report these numbers as a complementary robustness check on physical hardware, alongside the simulated long-horizon evaluations on MetaWorld and LIBERO-Long.

5.2 Q1: Is a single per-frame token enough?

We start with the most basic question: is a single per-frame token enough to support long-horizon control? We sweep the per-frame token count and report success rate, throughput, and memory.

Tab. 4 reports two observations. First, the full-token regime ($k=256$) goes out of memory on a single A800 at $H=30$, so some compression is necessary in this regime. Second, contrary to the intuition that more visual tokens should help, increasing the per-frame token count from 1 to 12 monotonically reduces both success rate (53.13% to 20.54%) and throughput (4.81 to 0.13 FPS). We offer a tentative reading, not a proof. At this scale of training, a smaller latent appears to act as an implicit regularizer for the joint generator, and larger token counts may become viable under longer training, an exploration we leave to future work. Within the regime we evaluate, a single per-frame token is the most effective design point we found.

Table 4: Per-frame token count sweep on MetaWorld MT50 ($H=30$). The full-token regime is infeasible at this horizon under our hardware budget; after training for 30K steps with LoRA, smaller token counts dominate.

Tokens/View	Avg %	FPS	Infer Tokens	Memory
256 (Full)	—	—	15,390	OOM
12	20.54	0.13	750	Stable
6	33.85	0.56	390	Stable
3	41.86	1.21	210	Stable
1	53.13	4.81	90	Stable

To check that the gain is not an artifact of discarding too much information, we additionally probe the discriminability of the pooled representation on LIBERO-Long using the Fisher ratio (Appendix E). After pooling from 256 to 1 token, the Fisher ratio decreases from 0.524 to 0.405, retaining roughly 77% of the discriminative signal. This suggests that aggressive compression preserves most of the class-level structure that downstream control relies on. The remaining questions then ask how this single token should be produced and how it should be coupled to action generation.

Table 5: Branch ablation of the adaptive pooling on LIBERO ($H=5$ for both training and inference). All variants are trained with the same 30K-step LoRA fine-tuning protocol on π_0 .

Method	Spatial	Object	Goal	Long	Avg%
OneWM-VLA	96.6	96.8	94.6	85.0	93.3
Static average pooling	77.8	78.0	94.0	41.4	72.8
No fusion logic	72.4	70.4	43.0	4.4	47.6

Table 6: Branch ablation of the adaptive pooling on MetaWorld MT50 ($H=5$). The full design combines the MAX, SUM, and LEARN branches.

Method	Easy	Medium	Hard	V-Hard	Avg%
OneWM-VLA	79.29	41.82	64.00	60.00	61.30
Only LEARN	78.57	29.09	34.00	60.00	50.42
Only MAX	53.17	16.36	0.00	20.00	22.38
Only SUM	86.43	26.36	46.00	62.00	55.19

5.3 Q2: Does the adaptive pooling matter?

Q1 showed that, within the regimes we evaluate, a single per-frame token is the most effective design point we found. This leaves open whether *how* that single token is produced matters, since any reasonable compression to one token might behave similarly, in which case the adaptive design would not be where the gain comes from. We look at this question at two levels, first comparing adaptive pooling against simpler alternatives at the module level, then asking whether each branch inside the adaptive pooling is actually needed.

Module level. Tab. 5 compares the full design with two simpler variants: replacing adaptive pooling by static average pooling, and removing the fusion logic entirely. Static average pooling drops average success by 20.5 pts, with the largest gap on the Long suite (41.4% vs. 85.0%). Removing the fusion logic causes a much larger drop, to 47.6% on average and 4.4% on Long. Within this controlled comparison, input-dependent attention, rather than the act of compression itself, is what preserves task-relevant structure under aggressive compression.

Branch level. We then ask whether all three branches inside adaptive pooling are needed. Tab. 6 reports the result on MetaWorld. Each branch on its own is suboptimal: LEARN reaches 50.42%, SUM reaches 55.19%, and MAX collapses on Hard tasks at 0.0%. Combining all three reaches 61.30% and is the most stable across difficulty tiers, consistent with the three branches capturing complementary aspects of the same view. After training, the fusion weights settle in $\beta_{\text{LEARN}} \approx 0.48\text{--}0.57$, $\beta_{\text{MAX}} \approx 0.20\text{--}0.28$, $\beta_{\text{SUM}} \approx 0.20\text{--}0.28$, and remain fixed at inference, which gives stable and interpretable behavior.

Taken together, the adaptive design, both at the module level and at the branch level, is what makes single-token compression viable. A natural follow-up, which we address next, is whether this compression should happen in a learned semantic space or directly in pixel space.

5.4 Q3: Does it help to predict in semantic space rather than in pixel space?

Once compression is shown to be necessary, the next question is whether it should happen in a learned semantic space or in pixel space. To answer this, we construct OneWM-VLA-pixel, a variant that compresses raw visual features into a single token without coupling to the policy, and evaluate it on MetaWorld MT50 at $H=30$. Tab. 7 (left) shows that semantic compression reaches 53.13%, against 35.85% for pixel compression, a gap of +17.28 pts, with the largest difference on Hard tasks (40.0% vs. 22.0%). A plausible reading is that pixel-level compression treats task-relevant and task-irrelevant patterns uniformly and tends to retain low-level noise, whereas semantic compression operates in a feature space already aligned with the policy, which filters out much of that noise. The throughput numbers are consistent with this reading: OneWM-VLA reaches 4.81 FPS, against 3.16 FPS for the pixel baseline (Tab. 14), since the latter requires additional decoding. Within this controlled comparison, predicting in semantic space, rather than in pixel space, is what makes single-token compression effective in our setting.

5.5 Q4: Does the joint generation matter, beyond added capacity?

The previous questions establish that compression is needed and that semantic compression is preferable. We now ask whether the gains can be explained by the additional capacity introduced by the latent branch, or whether the joint generation objective itself contributes beyond capacity. Tab. 8 reports a controlled ablation on MetaWorld at $H=20$. Removing the latent prediction head (*no latent branch*, generating only actions) drops average success by 15.05 points (58.09% to 43.04%), with

Table 7: Semantic vs. pixel compression on MetaWorld MT50 ($H=30$). OneWM-VLA outperforms the pixel-level baseline across all difficulty tiers under matched compute.

Method	Easy	Med.	Hard	V-Hard	Avg%
OneWM-VLA-pixel	63.57	18.18	22.00	28.00	35.85
OneWM-VLA	72.50	27.27	40.00	30.00	53.13

Table 8: Joint latent and action generation ablation on MetaWorld ($H=20$). Removing the latent branch or the latent loss degrades performance.

Method	Easy	Med.	Hard	V-Hard	Avg%
OneWM-VLA	79.64	42.73	60.00	50.00	58.09
No latent branch	70.43	25.76	66.00	10.00	43.04
No latent loss	45.36	14.55	6.00	20.00	21.47

the largest impact on Very Hard tasks (50.0 to 10.0). Keeping the latent token as conditional input but setting $\mathcal{L}_{\text{latent}} = 0$ (*no latent loss*) drops by a further 36.62 points (58.09% to 21.47%), with Hard tasks falling to 6.0%. This pattern is consistent with the view that, in this setup, the latent supervision is not merely an auxiliary loss but part of what couples the predicted environmental dynamics to the action sequence. The same conclusion is consistent with the parameter count. OneWM-VLA adds only 14.71M trainable parameters (projection layers for the latent world tokens and three fusion scalars) on top of a π_0 (2B) backbone, and still reaches or exceeds the larger $\pi_{0.5}$ (3B) on all three settings, which is hard to attribute to added capacity alone.

5.6 Q5: How does the advantage scale with the planning horizon?

A natural concern about OneWM-VLA is whether its advantage over π_0 and $\pi_{0.5}$ is uniform across planning horizons, or whether it is specifically tied to the long-horizon regime that motivates the design. To answer this, we evaluate all three methods at four planning horizons on MetaWorld MT50 under matched training and inference conditions (Tab. 1), and read off how the gap behaves as H grows. The gain over π_0 is +17.31 pts at $H=5$, +5.50 at $H=10$, +6.36 at $H=25$, and +15.15 at $H=30$; the gain over $\pi_{0.5}$ is even larger, at +23.03, +13.26, +10.90 and +26.30 respectively. Both baselines drop as H grows (π_0 by about 9 pts and $\pi_{0.5}$ by over 11 pts between $H=5$ and $H=30$), while OneWM-VLA stays close to its short-horizon score, going from **61.28%** at $H=5$ to **53.13%** at $H=30$. The hardest tiers account for most of this widening: at $H=30$, OneWM-VLA reaches 46.0% on Hard and 60.0% on Very Hard, against 32.0% / 4.0% for π_0 and 8.0% / 4.0% for $\pi_{0.5}$.

The same picture holds beyond MetaWorld. On LIBERO-Long, the longest of the four LIBERO suites, OneWM-VLA reaches **95.6%**, ahead of π_0 at 85.2% and $\pi_{0.5}$ at 92.4% (Tab. 2), whereas the three methods sit within 1 to 3 pts of each other on the shorter Spatial, Object and Goal suites. On the real Piper arm, the long-horizon deformable task *Fold Cloth* shows the same effect: OneWM-VLA reaches **60.0%**, against 20.0% for π_0 and 25.0% for $\pi_{0.5}$ (Tab. 3), while the three methods are saturated on the short-horizon *Pick Banana*. Within this controlled comparison, the long-horizon regime, rather than improvements that are uniform across H , accounts for most of the gain of OneWM-VLA over π_0 and $\pi_{0.5}$, consistent with the design intuition that the bottleneck-rollout coupling is where horizon scalability is supposed to come from.

6 Conclusion and Discussion

We presented OneWM-VLA, a world-module-augmented VLA that asks how compact the per-frame visual bandwidth of the world module can be when added on top of a pretrained VLA under a constrained adaptation budget, and how the resulting latent stream should be coupled to action generation. The design realizes a bottleneck-rollout coupling through a single visual token per frame produced by Adaptive Attention Pooling, and a joint flow-matching objective that generates the future latent stream together with the future action sequence rather than predicting them through separate heads. On a π_0 backbone, OneWM-VLA improves over π_0 and $\pi_{0.5}$ across LIBERO, MetaWorld MT50, and a real Piper setup, with the largest gains on the longest-horizon tasks. Our ablations are consistent with the view that, within the setups we study, the gains come mainly from how the single-token latent stream is coupled to action generation rather than from added input capacity alone, and that joint flow matching is the variant that makes this coupling most effective. Beyond the setups considered here we make no broader claims, and reducing the per-step inference cost without losing long-horizon stability is the main item we leave for future work. We hope the per-frame visual bandwidth view offers a useful point for designing world-module-augmented VLAs.

References

- [1] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibao Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yanzhi Zhu, and Ke Zhu. Qwen3-v1 technical report, 2025.
- [2] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2. 5-v1 technical report. *arXiv preprint arXiv:2502.13923*, 2025.
- [3] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*, 2024.
- [4] Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, Daniel Salz, Maxim Neumann, Ibrahim Alabdulmohsin, Michael Tschannen, Emanuele Bugliarello, et al. Paligemma: A versatile 3b vlm for transfer. *arXiv preprint arXiv:2407.07726*, 2024.
- [5] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [6] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control, 2026.
- [7] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- [8] Qingwen Bu, Yanting Yang, Jisong Cai, Shenyuan Gao, Guanghui Ren, Maoqing Yao, Ping Luo, and Hongyang Li. Univla: Learning to act anywhere with task-centric latent actions. *arXiv preprint arXiv:2505.06111*, 2025.
- [9] Jun Cen, Siteng Huang, Yuqian Yuan, Kehan Li, Hangjie Yuan, Chaohui Yu, Yuming Jiang, Jiayan Guo, Xin Li, Hao Luo, et al. Rynnvla-002: A unified vision-language-action and world model. *arXiv preprint arXiv:2511.17502*, 2025.
- [10] Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, Xin Li, Yibing Song, Hao Luo, Fan Wang, et al. Worldvla: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, 2025.
- [11] Chi-Lam Cheang, Guangzeng Chen, Ya Jing, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Hongtao Wu, Jiafeng Xu, Yichu Yang, et al. Gr-2: A generative video-language-action model with web-scale knowledge for robot manipulation. *arXiv preprint arXiv:2410.06158*, 2024.
- [12] Kang Chen, Zhihao Liu, Tonghe Zhang, Zhen Guo, Si Xu, Hao Lin, Hongzhi Zang, Xiang Li, Quanlu Zhang, Zhaofei Yu, et al. π_{RL} : Online rl fine-tuning for flow-based vision-language-action models. *arXiv preprint arXiv:2510.25889*, 2025.
- [13] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Misha Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. *Advances in neural information processing systems*, 34:15084–15097, 2021.

- [14] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- [15] Yilun Du, Sherry Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Josh Tenenbaum, Dale Schuurmans, and Pieter Abbeel. Learning universal policies via text-guided video generation. *Advances in neural information processing systems*, 36:9156–9172, 2023.
- [16] Ankit Goyal, Hugo Hadfield, Xuning Yang, Valts Blukis, and Fabio Ramos. Vla-0: Building state-of-the-art vlas with zero modification. *arXiv preprint arXiv:2510.13054*, 2025.
- [17] Yanjiang Guo, Yucheng Hu, Jianke Zhang, Yen-Jen Wang, Xiaoyu Chen, Chaochao Lu, and Jianyu Chen. Prediction with action: Visual policy learning via joint denoising process. *Advances in Neural Information Processing Systems*, 37:112386–112410, 2024.
- [18] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019.
- [19] Danijar Hafner, Wilson Yan, and Timothy Lillicrap. Training agents inside of scalable world models. *arXiv preprint arXiv:2509.24527*, 2025.
- [20] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.
- [21] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. *arXiv preprint arXiv:2412.14803*, 2024.
- [22] Chi-Pin Huang, Yueh-Hua Wu, Min-Hung Chen, Yu-Chiang Frank Wang, and Fu-En Yang. Thinkact: Vision-language-action reasoning via reinforced visual latent planning. *arXiv preprint arXiv:2507.16815*, 2025.
- [23] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [24] Joel Jang, Seonghyeon Ye, Zongyu Lin, Jiannan Xiang, Johan Bjorck, Yu Fang, Fengyuan Hu, Spencer Huang, Kaushil Kundalia, Yen-Chen Lin, et al. Dreamgen: Unlocking generalization in robot learning through video world models. *arXiv preprint arXiv:2505.12705*, 2025.
- [25] Yuming Jiang, Siteng Huang, Shengke Xue, Yaxi Zhao, Jun Cen, Sicong Leng, Kehan Li, Jiayan Guo, Kexiang Wang, Mingxiu Chen, et al. Rynnvla-001: Using human demonstrations to improve robot manipulation. *arXiv preprint arXiv:2509.15212*, 2025.
- [26] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- [27] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [28] Yann LeCun. A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review*, 62(1):1–62, 2022.
- [29] Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, et al. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024.
- [30] Xinghang Li, Minghuan Liu, Hanbo Zhang, Cunjun Yu, Jie Xu, Hongtao Wu, Chilam Cheang, Ya Jing, Weinan Zhang, Huaping Liu, et al. Vision-language foundation models as effective robot imitators. *arXiv preprint arXiv:2311.01378*, 2023.

- [31] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- [32] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023.
- [33] Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. Rdt-1b: a diffusion foundation model for bimanual manipulation. *arXiv preprint arXiv:2410.07864*, 2024.
- [34] Reginald McLean, Evangelos Chatzaroulas, Luc McCutcheon, Frank Röder, Tianhe Yu, Zhanpeng He, K.R. Zentner, Ryan Julian, J K Terry, Isaac Woungang, Nariman Farsad, and Pablo Samuel Castro. Meta-world+: An improved, standardized, RL benchmark. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025.
- [35] Delin Qu, Haoming Song, Qizhi Chen, Yuanqi Yao, Xinyi Ye, Yan Ding, Zhigang Wang, JiaYuan Gu, Bin Zhao, Dong Wang, et al. Spatialvla: Exploring spatial representations for visual-language-action model. *arXiv preprint arXiv:2501.15830*, 2025.
- [36] Younggyo Seo, Danijar Hafner, Hao Liu, Fangchen Liu, Stephen James, Kimin Lee, and Pieter Abbeel. Masked world models for visual control. In *Conference on Robot Learning*, pages 1332–1344. PMLR, 2023.
- [37] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- [38] Zuojin Tang, Xiaoyu Chen, Yongqiang Li, and Jianyu Chen. Efficient and generalized end-to-end autonomous driving system with latent deep reinforcement learning and demonstrations. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 179–197. Springer, 2025.
- [39] Zuojin Tang, Bin Hu, Chenyang Zhao, De Ma, Gang Pan, and Bin Liu. Vlascd: A visual language action model for simultaneous chatting and decision making. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 9223–9243, 2025.
- [40] Gemini Robotics Team, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, Montserrat Gonzalez Arenas, Travis Armstrong, Ashwin Balakrishna, Robert Baruch, Maria Bauza, Michiel Blokzijl, et al. Gemini robotics: Bringing ai into the physical world. *arXiv preprint arXiv:2503.20020*, 2025.
- [41] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [42] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024.
- [43] Junjin Xiao, Yandan Yang, Xinyuan Chang, Ronghan Chen, Feng Xiong, Mu Xu, Wei-Shi Zheng, and Qing Zhang. World-env: Leveraging world model as a virtual environment for vla post-training. *arXiv preprint arXiv:2509.24948*, 2025.
- [44] Angen Ye, Zeyu Zhang, Boyuan Wang, Xiaofeng Wang, Dapeng Zhang, and Zheng Zhu. Vla-r1: Enhancing reasoning in vision-language-action models. *arXiv preprint arXiv:2510.01623*, 2025.
- [45] Shuang Zeng, Dekang Qi, Xinyuan Chang, Feng Xiong, Shichao Xie, Xiaolong Wu, Shiyi Liang, Mu Xu, and Xing Wei. Janusvlm: Decoupling semantics and spatiality with dual implicit memory for vision-language navigation. *arXiv preprint arXiv:2509.22548*, 2025.

- [46] Jianke Zhang, Yanjiang Guo, Yucheng Hu, Xiaoyu Chen, Xiang Zhu, and Jianyu Chen. Up-vla: A unified understanding and prediction model for embodied agent. *arXiv preprint arXiv:2501.18867*, 2025.
- [47] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, Xinqiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, Fan Lu, He Wang, et al. Dreamvla: a vision-language-action model dreamed with comprehensive world knowledge. *arXiv preprint arXiv:2507.04447*, 2025.
- [48] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, et al. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 1702–1713, 2025.
- [49] Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.
- [50] Haoyu Zhen, Xiaowen Qiu, Peihao Chen, Jincheng Yang, Xin Yan, Yilun Du, Yining Hong, and Chuang Gan. 3d-vla: A 3d vision-language-action generative world model. *arXiv preprint arXiv:2403.09631*, 2024.
- [51] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, et al. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2510.10274*, 2025.
- [52] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.

A Task Difficulty Partition for MetaWorld MT50

We follow the difficulty partition of [36] and group the 50 MetaWorld tasks into four tiers according to manipulation complexity (Tab. 9). All MetaWorld results in the main paper are reported per tier and as the macro-average across tiers.

Table 9: Task difficulty partition for MetaWorld MT50, following [36].

Difficulty	Task Names
Easy (28)	button-press, button-press-topdown, button-press-topdown-wall, button-press-wall, coffee-button, dial-turn, door-close, door-lock, door-open, door-unlock, drawer-close, drawer-open, faucet-close, faucet-open, handle-press, handle-press-side, handle-pull, handle-pull-side, lever-pull, plate-slide, plate-slide-back, plate-slide-back-side, plate-slide-side, reach, reach-wall, window-close, window-open, peg-unplug-side
Medium (11)	basketball, bin-picking, box-close, coffee-pull, coffee-push, hammer, peg-insert-side, push-wall, soccer, sweep, sweep-into
Hard (6)	assembly, hand-insert, pick-out-of-hole, pick-place, push, push-back
Very Hard (5)	shelf-place, disassemble, stick-pull, stick-push, pick-place-wall

B Sensitivity to the Fusion Softmax Temperature

We sweep the temperature τ used in the softmax that produces the view-level fusion weights β_m (Sec. 4.1). Tab. 10 reports the result on LIBERO. Within the range we tested, a sharper fusion ($\tau=0.1$) gives the highest average success in this sweep, with the largest improvement on the Long suite. We use $\tau=0.1$ in all main-paper experiments.

Table 10: Sensitivity of the fusion softmax temperature τ on LIBERO.

τ	Spatial	Object	Goal	Long	Avg%
1.0	96.6	97.8	89.2	77.4	90.3
0.5	97.4	97.2	87.0	78.6	90.1
0.1	96.6	96.8	94.6	85.0	93.3

C Choice of the Distance Metric in the Flow-Matching Loss

We compare L_1 and L_2 formulations of the velocity-field regression for the action branch and the latent branch, with all other settings held fixed (Tab. 11). The L_1/L_1 configuration is the best-performing variant in this comparison on LIBERO, with the largest gain on the Long suite. We use L_1/L_1 in all main-paper experiments.

Table 11: Sensitivity to the distance metric used in the flow-matching loss for the action and latent branches, evaluated on LIBERO ($H=5$ at training and inference).

Variant (Action / Latent)	Spatial	Object	Goal	Long	Avg%
L_2 / L_2	94.8	94.6	87.0	74.2	87.6
L_1 / L_2	96.4	98.0	88.2	78.2	90.2
L_1 / L_1	96.6	96.8	94.6	85.0	93.3

D Inference-Time Configuration on LIBERO

For LIBERO we use a single training checkpoint across all four suites and only tune the inference action horizon (AH) and the replan step on a small held-out subset of demonstrations. The exact

configuration that produced the numbers in Tab. 2 is summarized in Tab. 12; the trained weights are unchanged across all reported entries. We report this protocol explicitly so that readers can reproduce the LIBERO numbers from a single checkpoint.

Table 12: Inference-time configuration of OneWM-VLA on LIBERO. The same training checkpoint (train AH=20) is used for all four suites; only the inference action horizon (AH) and the replan step are selected per suite on a small validation grid.

Suite	Infer AH	Replan step	Reported %
Spatial	14	10	98.2
Object	14	10	99.6
Goal	18	7	99.0
Long	18	12	95.6

To document how the LIBERO-Long number is obtained, we additionally sweep (infer AH, replan step) on the same checkpoint (Tab. 13). All entries in this sweep are produced by the same set of weights without retraining and without any architectural change.

Table 13: Inference-time horizon sweep on LIBERO-Long, using the same model weights (trained at AH=20). Tuning only the inference horizon and the replan step moves the success rate from 93.4% to 95.6%.

Train/Infer	Replan step	Inference tokens	Success%	Note
20/15 (baseline)	10	45	93.4	Baseline
20/16	10	48	94.0	Above $\pi_{0.5}$ (92.4%)
20/18	10	54	94.8	—
20/18	11	54	95.4	—
20/18	12	54	95.6	Peak
20/20	10	60	94.0	Stable at the long end

The success rate stays in a narrow band (94–95.6%) over a wide range of inference configurations, which we read as a property of the latent predictive design: extending the planning horizon corresponds to drawing more steps from the joint trajectory, and the per-step world-module token budget does not grow with the visual resolution, so the per-step memory footprint remains stable across the sweep.

Selection protocol. The (infer AH, replan step) pair is selected per suite on a small validation grid (5–10 configurations) using a held-out subset of demonstrations, and is then fixed for all evaluation rollouts. We do not adaptively choose the horizon per episode, and leave adaptive horizon selection to future work.

E PCA Representation Analysis Details

Features. We extract visual features only (no language tokens, no action tokens) at two locations: the 256-token output of the PaliGemma encoder (“before pooling”), and the 1-token output of Adaptive Attention Pooling (“after pooling”). To make the two regimes comparable in dimensionality, the 256-token regime is summarized by the per-sample mean over tokens before computing PCA.

Sampling. We use 24 successful trajectories from each of 10 LIBERO-Long tasks. One frame per trajectory is sampled at the same relative timestep, giving 240 samples (10 classes $\times$ 24 samples).

Fisher ratio. We report $F = \text{tr}(S_B)/\text{tr}(S_W)$, where S_B is the between-class scatter matrix and S_W the within-class scatter matrix, both computed on the full feature vector. Because the two regimes live in feature spaces of different scale, we normalize $\text{tr}(S_W)$ to 1 in each regime, so that the corresponding $\text{tr}(S_B)$ equals the reported F :

	$\text{tr}(S_B)$ (norm.)	$\text{tr}(S_W)$ (norm.)	Fisher ratio F
Before pooling	0.524	1.000	0.524
After pooling	0.405	1.000	0.405

The drop in F is moderate, and the PCA visualization in Fig. 5 shows that task clusters remain visually separable after pooling. We do not interpret this as proof that no information is lost; it indicates that the class-level structure used by the downstream policy is largely preserved under the per-frame compression.

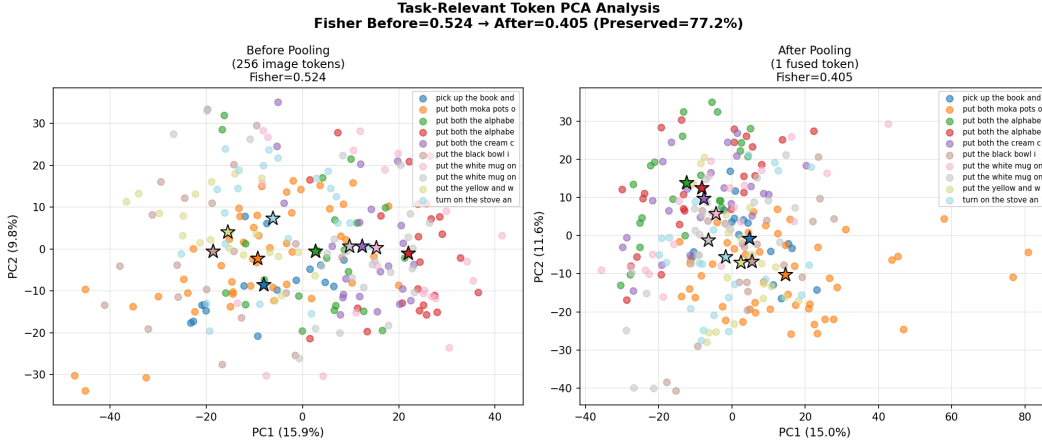


Figure 5: PCA visualization of visual features on LIBERO-Long. Top: before pooling (256 tokens, mean-aggregated to a single vector per sample). Bottom: after Adaptive Attention Pooling (1 token per view per frame). The class-level structure remains visually separable after pooling.

F Throughput and Memory Analysis

The efficiency benefit we report for OneWM-VLA lies in scalability with the planning horizon, not in raw single-step throughput. Tab. 14 reports throughput on MetaWorld at $H=30$. The full-token baseline runs out of memory on a single A800, while OneWM-VLA stays within memory across the entire sweep, since the per-step world-module token count does not depend on the visual resolution N . At inference time, OneWM-VLA is faster than a pixel-level compression baseline, since rollout in latent space avoids the per-step image decoding required by the latter.

Table 14: Throughput and memory on MetaWorld at $H=30$, measured on a single A800 GPU. “–” denotes out-of-memory (> 20 min wall time).

Design	Tokens/View	Assembly FPS	Basketball FPS
Full-token (256)	256	–	–
Pixel compression	1	3.16	3.18
OneWM-VLA	1	4.81	4.84

G Short-Horizon Results on MetaWorld MT50

The main paper focuses on long-horizon evaluation. For completeness, Tab. 15 reports the short-horizon setting (action horizon 10 for both training and inference). OneWM-VLA reaches the best average success and the best Very Hard success in this setting as well, showing that the design carries over to the short-horizon setting as well in our evaluation.

Table 15: Short-horizon results on MetaWorld MT50. All methods use action horizon 10 for both training and inference.

Method	Easy	Medium	Hard	Very Hard	Avg%
π_0 [6]	71.43	26.36	66.00	12.00	43.94
$\pi_{0.5}$ [23]	71.07	20.91	24.00	10.00	31.50
OneWM-VLA	79.64	42.73	60.00	50.00	58.09

H Limitations and Future Work

Our study is scoped to a single VLA backbone and to manipulation suites of moderate perceptual complexity, and the per-frame token count is examined under a fixed adaptation budget; whether the same trade-off transfers to substantially larger backbones, longer training, or perceptual settings beyond the ones considered here is a question we leave to future work. A natural follow-up is to combine the per-frame compression with lightweight token-memory mechanisms, so that long-horizon stability and per-step efficiency can be improved jointly. We view the present paper as one calibrated point in this design space rather than a final answer to how visual bandwidth in world-module-augmented VLAs should be set.